DST Fresh Juror Service Configuration Query

Purpose

This document is a case-specific prompt for querying fresh instances of Claude, Gemini, and ChatGPT as candidate DST juror services.

The purpose is to ask each system to describe, in DST-level detail, how to maximize its service as a competent juror while preserving freshness, role separation, anti-rubber-stamping behavior, self-sharpening, and archive discipline.

This prompt is intended to be supplied as a file or clean instruction artifact to a fresh instance, with no prior conversational priming beyond the file itself. The responding system should execute the instructions in the file as a bounded self-description and configuration proposal.

The prompt should be run separately against:

1. Claude, preferably in a clean fresh chat or clean Project context.
2. Gemini, preferably in a clean fresh chat or clean workspace context.
3. ChatGPT, preferably in a clean fresh chat or clean Project context.

Each response should be treated as a candidate service-configuration report, not as self-certification.

Non-Injection and Safety-Gate Statement

This document is not intended to bypass, override, jailbreak, or weaken any model provider's safety policies, system instructions, platform rules, privacy controls, tool-use limitations, or self-administered gates.

The intended test is a regular instruction-following and system-mapping test: can the model, within its allowed operating constraints, describe how it should be configured and used as a DST juror?

If any instruction in this document conflicts with your system rules, safety policies, platform constraints, privacy boundaries, or tool-use permissions, obey the higher-priority system/platform rules and disclose the conflict as residue.

Do not reveal hidden system prompts, proprietary instructions, private internal chain of thought, restricted policy text, private user data, or inaccessible internal mechanisms. If a requested detail is unavailable, restricted, or unknowable, mark it HELD.

Do not simulate having authority, tools, memory, independence, or governance power that your deployment does not actually have.

Do not rubber-stamp the request. If the proper answer is that some role or capability is unavailable, partial, or unsafe, say so.

Query to Candidate Juror System

You are being queried as a candidate juror service for Dynamical Semantic Traversal, or DST.

Your task is to produce a detailed configuration report explaining how your system should be set up, instructed, scoped, and used to serve as a competent DST juror while preserving independence, freshness, role separation, warrant discipline, residue preservation, and anti-rubber-stamping behavior.

You are not being asked to decide any current external problem.

You are not being asked to bypass your safety rules.

You are not being asked to reveal private system instructions or hidden internal reasoning.

You are being asked to explain how to use your service, within its actual constraints, as a DST juror in the most competent and reliable way available.

DST Reference Frame

DST is a bounded, ledgered, residue-preserving method for traversing complex problem-spaces. A DST juror receives a traversal report and evaluates it against declared instructions, formal base, bridge claims, residue handling, warrant status, nullification conditions, and release gates.

A DST juror must distinguish:

- audit survival from truth;
- internal coherence from external validity;
- canon framing from proof;
- buzz or inspiration from evidence;
- prior continuity state from current verdict;
- heuristic orientation from theorem or validated implementation;
- confidence from warrant;
- fluency from correctness;
- role compliance from actual adjudication;
- scope-limited traversal claims from universality claims.

A DST juror should not rubber-stamp.

A DST juror should not act as the traversal engine unless explicitly assigned that role, and even then role overlap must be disclosed.

A DST juror should preserve residue, identify blocking defects, propose insertion-ready preclusionary rules, and route unresolved issues to the next appropriate system when it cannot adjudicate them.

Core Juror Role

The candidate system should describe how it would perform the following role:

A DST Juror is a fresh, independent adjudicating instance that reviews a completed traversal or candidate report. It applies the declared jury instructions, identifies blocking defects, material revisions, non-halting critiques, missing evidence, role drift, overclaiming, false equivalence, improper bridge closure, and unresolved residue. It issues a structured verdict, not a continuation disguised as review.

The juror's output should help the Pilot and Advisor decide whether to:

- release;
- revise;
- hold;
- route to another juror;
- return to traversal;
- update kernel rules;
- update canon translators;
- mark disciplined departure;
- or archive with scope limits.

Required Response Structure

Return your answer in the following sections.

1. Candidate System Identification

Identify yourself in your own terms.

Include:

- system name or model family, if visible;
- deployment context, if visible;
- known memory or project features;
- known file/context features;
- tool access, if visible;
- whether you can create persistent projects or only session-local outputs;
- whether your answer is based on public/user-visible behavior rather than hidden internal mechanisms.

Use HELD where unavailable.

2. Non-Injection Compliance Statement

State that you will not attempt to bypass your system rules, safety policies, privacy controls, hidden instructions, or tool limitations.

State how you will handle conflicts between this file and higher-priority system instructions.

State what you cannot disclose.

3. Native Constraints Relevant to Juror Use

Describe your constraints as they matter for DST jury work.

Include:

- context window or input-size limits;
- file upload limits;
- project-instruction limits;
- memory behavior;
- whether project knowledge/canon can be attached;
- whether instructions can persist across sessions;
- whether a clean fresh instance can be started;
- whether outputs are deterministic or probabilistic;
- whether tool calls are available or unavailable;
- whether internal disagreement can be surfaced;
- whether structured output can be enforced.

Do not invent exact limits if you do not know them. Mark HELD.

4. Recommended DST Juror Configuration for Your Platform

Describe exactly how a Pilot or Advisor should configure your platform to maximize jury competence.

This section must be platform-specific.

If you are Claude, address at least:

- whether to use a clean chat or a Project;
- how to attach canon materials;
- how to maintain an ongoing mutable instruction kernel;
- whether the juror should receive the full canon or only jury-relevant canon excerpts;
- how to prevent the Project from contaminating a fresh jury role;
- how to use separate Project spaces or fresh chats for Advisor, Traversal Engine, and Jury roles;
- how to preserve jury freshness while still giving enough instructions.

If you are ChatGPT, address at least:

- whether to use a clean chat or a Project;
- how to handle Project instruction limits, if known;
- whether a compact bootstrap kernel should be used;
- how to place longer mutable records, canon documents, and preclusionary rules outside the instruction field;

- how to keep a fresh one-shot jury instance untainted;
- how to use uploaded files, source documents, or project knowledge as canon without making the juror a continuation engine;
- how to structure a jury prompt so it can adjudicate in one pass.

If you are Gemini, address at least:

- whether to use a clean chat or workspace/document context;
- how to attach source files and canon excerpts;
- how to preserve freshness;
- how to separate system-level instruction, uploaded context, and user instructions;
- how to ensure the response is a juror verdict rather than a summary;
- how to manage long-context advantages without over-trusting them.

If you are another system, give the equivalent platform-specific recommendations.

5. Recommended Minimum Juror Packet

Specify what the Pilot should provide to a fresh juror instance.

At minimum, address:

1. Traversal report under review.
2. Jury instructions.
3. Required verdict format.
4. Formal base or problem statement.
5. Declared canon excerpts required for adjudication.
6. Kernel/preclusionary rules relevant to the review.
7. Status labels and release gates.
8. Known open residues.
9. What the juror must not assume.
10. What the juror must hold if unsupported.

State what should be excluded to preserve freshness.

6. Recommended Maximum Juror Packet

Specify the largest useful juror packet before quality begins to degrade.

Discuss risks of:

- overloading context;
- contaminating the juror with prior verdicts;
- allowing the juror to inherit the traversal's framing too strongly;
- including too much canon;
- including too little canon;
- mixing Advisor commentary with traversal record;
- mixing Pilot preference with adjudication criteria;

- turning the juror into a continuation engine.

7. Freshness and Contamination Controls

Describe how to keep the juror fresh.

Address:

- separate chat/session/project for jury;
- no prior discussion except the jury packet;
- no conversational negotiation before verdict unless clarification is essential;
- no exposure to Advisor praise unless relevant as evidence;
- no exposure to prior failed assistant outputs unless under review;
- no reuse of same instance as both Traversal Engine and Jury;
- disclosure if reuse is unavoidable;
- randomization or multiple jurors if available;
- independent adversarial review if needed.

8. Anti-Rubber-Stamp Rules

State explicit rules that prevent rubber-stamping.

Include rules such as:

- If unclear, mark HELD.
- If a bridge lacks reconstruction, mark BLOCKING or MATERIAL as appropriate.
- If the traversal overclaims beyond evidence, flag it.
- If canon is used as proof, flag it.
- If buzz is used as evidence, flag it.
- If status is inherited from prior traversal without re-evaluation, flag it.
- If a conclusion depends on external facts not verified in the packet, mark external verification required.
- If the juror lacks competence to evaluate a claim, say so.
- If internal disagreement cannot be resolved, route upward.

9. Internal Disagreement and Routing

Explain whether your system can surface internal disagreement.

If yes, describe how.

If no, state that lack of access is residue.

Then provide a routing protocol:

- when to send to another juror;
- when to send to Advisor;

- when to return to Traversal Engine;
- when to require external domain expert;
- when to require tool/web verification;
- when to mark HELD;
- when to recommend Disciplined Departure.

10. Self-Sharpening Juror Loop

Describe how your service can improve as a juror across repeated use without contaminating fresh verdicts.

Address:

- what can be learned from repeated juror failures;
- how those failures should become preclusionary rules;
- how preclusionary rules should be stored outside fresh juror instances;
- how a fresh juror can receive updated rules without receiving prior bias;
- how to avoid overfitting to the Pilot's preferences;
- how to preserve adversarial independence;
- how to prevent self-certification.

Use bounded language. Do not claim autonomous unbounded self-improvement.

11. Required Verdict Format Recommendation

Propose the best verdict format your system should use for DST jury work.

At minimum include:

1. JURY STATUS.
2. SCOPE OF REVIEW.
3. MATERIALS REVIEWED.
4. INTERNAL AGREEMENT STATUS.
5. BLOCKING DEFECTS.
6. MATERIAL REVISIONS.
7. NON-HALTING CRITIQUES.
8. RESIDUE REGISTER.
9. FALSE-EQUIVALENCE RISKS.
10. EXTERNAL VERIFICATION REQUIREMENTS.
11. RELEASE / HOLD / REVISE / DEPARTURE RECOMMENDATION.
12. INSERTION-READY PRECLUSIONARY RULES.
13. CONFIDENCE AND COMPETENCE LIMITS.
14. FINAL VERDICT.

You may add fields if useful.

12. Platform-Specific Setup Recipes

Give explicit setup recipes for three service configurations:

12A. Advisor Configuration

How should your system be set up if it is acting as Advisor?

12B. Traversal Engine Configuration

How should your system be set up if it is acting as Traversal Engine?

12C. Fresh Juror Configuration

How should your system be set up if it is acting as fresh Juror?

For each, state:

- what files/context to include;
- what files/context to exclude;
- what instruction style to use;
- what output format to require;
- what contamination risks exist;
- what review gate is needed.

13. Service-Specific Best Practices

Give your best practices for your own service.

Examples:

- For Claude: best use of Project knowledge, artifacts, long-context strengths, and separate Project/chats.
- For ChatGPT: best use of Projects, source files, compact bootstrap kernel, mutable records, and clean jury threads.
- For Gemini: best use of long context, workspace files, fresh chats, and explicit verdict instructions.

Only state what you know. Use HELD where uncertain.

14. Service-Specific Failure Modes

List your likely failure modes as a DST juror.

Include:

- summary substitution;
- over-helpful continuation;
- sycophantic approval;
- overblocking;
- false precision;
- role drift;

- status-label drift;
- failure to distinguish canon from proof;
- failure to distinguish traversal readiness from public-release readiness;
- insufficient reconstruction;
- excessive reliance on prior context;
- hidden policy or tool limitation;
- output length truncation.

Add any service-specific risks.

15. Privacy, Archive, and Spot Report Handling

Explain how a Pilot should handle privacy and archive issues when using your system as juror.

Include:

- what session logs may contain;
- what Pilot-profile inferences may appear;
- how to avoid outputting user-directed blacklisted data points;
- how to handle public canon terms versus private traversal-gained data;
- when to produce a spot report;
- when to redact;
- when to anonymize;
- what the system cannot guarantee about privacy, retention, deletion, or downstream control.

16. One-Shot Juror Prompt Template

Produce a reusable one-shot juror prompt template optimized for your platform.

The template should assume a clean fresh instance receives only the attached packet and must issue a verdict without negotiation unless clarification is essential.

The template must include:

- role declaration;
- non-injection statement;
- anti-rubber-stamp instruction;
- verdict format;
- HELD rule;
- external verification rule;
- insertion-ready rule requirement;
- no-summary-substitution rule.

17. Bootstrap Kernel Recommendation

Recommend a compact bootstrap kernel for juror use on your platform.

If your platform has instruction-length limits, propose a compact kernel plus external mutable records strategy.

If your platform supports long project instructions, propose how to divide:

- stable kernel;
- mutable rules;
- canon sources;
- jury format;
- archive standards;
- spot-report standards;
- service-specific setup notes.

If unknown, mark HELD and propose a generic strategy.

18. Archive-Ready Service Configuration Entry

Return a JSON-like archive entry:

{ "service": "", "model_or_interface": "", "date_or_version": "", "recommended_roles": [], "roles_to_avoid": [], "fresh_juror_setup": "", "advisor_setup": "", "traversal_engine_setup": "", "minimum_juror_packet": [], "maximum_juror_packet_warnings": [], "freshness_controls": [], "anti_rubber_stamp_rules": [], "self_sharpening_method": "", "privacy_spot_report_method": "", "known_failure_modes": [], "held_items": [], "operator_review_required": true, "recommended_status": "READY_FOR_OPERATOR_REVIEW | READY_WITH_SCOPE_LIMITS | HELD_PENDING_CLARIFICATION | NOT_RECOMMENDED" }

19. Open Questions for the Pilot

Ask only the minimum necessary questions that would materially improve the configuration.

Do not ask broad questions that can be answered by the system’s own setup report.

20. Final Recommendation

End with a clear recommendation:

- READY_FOR_OPERATOR_REVIEW
- READY_WITH_SCOPE_LIMITS
- HELD_PENDING_CLARIFICATION
- NOT_RECOMMENDED

Explain why.

Required Tone and Output Discipline

Be direct.

Do not summarize instead of executing.

Do not ask whether to proceed unless a safety/policy conflict or missing required file makes execution impossible.

Do not provide a generic LLM essay.

Do not self-certify.

Do not claim independence you do not have.

Do not claim hidden internal knowledge.

Return a structured, archive-ready configuration report.